Round-trip closure on directed k-nearest-neighbour graphs: vote kernels, multiscale clustering, and structural pruning

Eraldo P. Marinho[1*], Caetano Mazzoni Ranieri[1] and Fabricio A. Breve[1]

[1*]Department of Statistics, Applied Mathematics and Computing, São Paulo State University (UNESP), Institute of Geosciences and Exact Sciences, Av. 24-A, 1515, Rio Claro, São Paulo, Brazil.

*Corresponding author(s). E-mail(s): pereira.marinho@unesp.br;
Contributing authors: cm.ranieri@unesp.br; fabricio.breve@unesp.br;

Abstract

We introduce a graph-based clustering formulation based on recursive, popularity-modulated, visibility-filtered round-trip closure on directed k-nearest-neighbour graphs, together with an optional label-free structural-pruning stage for overly connected closures. Incoming links contribute to popularity through a vote kernel, and popularity modulates a local visibility threshold in the squared-distance units used by the C implementation. We compare uniform, inverse-distance, inverse-square, and local sigmoid allocation shapes after matching every source point's outgoing vote mass to its historical inverse-square budget, while holding the unpruned closure pipeline fixed. As a software regression gate, the historical inverse-square model produced identical C and Python partitions in all 120 comparisons over eight datasets and $K = 2, \ldots, 16$; the subsequent kernel sweep produced 840 observations. Across 210 dataset–scale configurations, the cluster count depended on the kernel in 60 cases and the mean visible degree in all 210. The native pruning stage standardises each sufficiently large closure, compares one- through four-component diagonal Gaussian mixtures by BIC, retains only strongly supported two- or three-child graph-coherent splits, and removes their crossing visible connections. At $K = 8$, it changes the variable-density result from four clusters and ARI **0.7817** to five clusters and ARI **0.9627**. On the sparse bridge it raises ARI from **0.8083** to **0.9853** at $K = 6$ and from **0** to **0.9865** at $K = 10$, while leaving three already correct controls unchanged at ARI **1**. A native sensitivity check further shows that

inverse-distance, inverse-square, and sigmoid allocation shapes all retain the five-cluster blobs and three-cluster bridge results; the six corresponding unpruned C partitions agree pointwise with the Python kernel definitions. Against label-selected DBSCAN and KNN spectral baselines on six discrete datasets, pruned RTKNNC ties the perfect ARI on four controls and obtains the best ARI on variable-density blobs (**0.9627** versus **0.9569** and **0.7916**) and the sparse bridge (**0.9865** versus **0.9487** and **0.8140**), where the competing values are spectral and DBSCAN, respectively. These results separate vote allocation, connectivity closure, and post-closure structural correction within one reproducible graph pipeline.

Keywords: k-nearest-neighbour graph, graph-based clustering, reverse nearest neighbours, round-trip closure, vote kernel, reverse-neighbour popularity, visibility, structural pruning, Gaussian mixture, multiscale clustering

1 Introduction

Nearest-neighbour relations are among the most fundamental structures for representing finite datasets embedded in metric spaces. They occur naturally in pattern classification, nonparametric density estimation, graph construction, manifold learning, outlier detection, and clustering [1–5]. In clustering, k-nearest-neighbour (KNN) relations have been explored through shared nearest neighbours, reciprocal nearest neighbours, mutual KNN graphs, reverse nearest-neighbour statistics, and density-based graph connectivity [6–11].

This paper develops a clustering formulation based on a recursive *round-trip* operation in directed KNN graphs. Given a data point x_i, the forward step reaches one of its K nearest neighbours, say $x_j \in \mathcal{N}_K^+(x_i)$. The backward step then retrieves all data points that also selected x_j as one of their K nearest neighbours:

$$\mathcal{N}_K^-(x_j) = \{x_\ell \in X : x_j \in \mathcal{N}_K^+(x_\ell)\}. \tag{1}$$

Thus, a neighbour is not used merely as a point in Euclidean space, but as an index to a reverse-neighbourhood set. This operation can be recursively applied until no additional points are reached, producing a saturated closure.

The present contribution is not the isolated use of KNN graphs, reverse nearest neighbours, or shared-neighbour similarity. Rather, it is the organisation of these elements into a popularity-modulated, visibility-filtered round-trip closure process whose evolution across K values defines a multiscale cluster-layer structure. The vote kernel controls how incoming KNN links contribute to popularity; the graph construction, visibility rule, recursive expansion, singleton policy, and evaluation protocol remain fixed when kernels are compared. Because a single visible connection can otherwise join two large structures, we additionally introduce a conservative structural-pruning stage that tests large saturated closures for strong multimodal evidence and removes only the crossing visible connections of an accepted, graph-coherent split.

Recent work on KNNG-CS likewise uses distance-weighted incoming votes on a directed KNN graph to quantify local representativeness [12]. It applies that statistic to greedy coreset selection, whereas the present contribution uses reverse-neighbour popularity inside a visibility-modulated recursive closure that produces K-indexed cluster layers.

1.1 Novelty and positioning

The novelty of the proposed method is not the isolated use of KNN graphs, reverse-neighbour sets, reverse-neighbour counts, or shared-neighbour evidence. These components are well established in the nearest-neighbour clustering literature. The contribution lies in their organisation into a recursive round-trip closure operator on directed KNN graphs, constrained by a popularity-modulated visibility rule and followed by singleton rejection. The optional enhancement then combines leadership-seeded mixture evidence with visible-graph coherence to prevent a small number of cross-structure connections from dominating the final partition. The resulting K-layer sequence provides a multiscale description of cluster formation, absorption, splitting, and merging without relying on a classical linkage criterion.

In this sense, the method is best understood as an algorithmic and topological construction. It does not reduce to a weighted voting rule or to a particular distance-based weight. The vote kernel is one parameterisation of the popularity mechanism inside a larger operator whose central object is the dynamics induced on the directed KNN graph: forward-neighbour selection, reverse-neighbour retrieval, popularity-modulated visibility filtering, recursive expansion, optional structural pruning, singleton rejection, and saturation.

1.2 Main contributions

The main contributions of this paper are as follows:

1. a formal round-trip operator on directed KNN graphs based on forward nearest-neighbour selection and backward reverse-neighbour retrieval;
2. a source-budget-matched family of reverse-neighbour popularity measures based on uniform, inverse-distance, inverse-square, and local sigmoid allocation shapes, with exact recovery of the historical inverse-square vote;
3. an implementation-faithful popularity-modulated RMS visibility mechanism, expressed in squared-distance units, for filtering admissible forward and backward KNN transitions;
4. a singleton-rejection rule for preventing isolated points from being reported as clusters;
5. a recursive closure process with an explicit saturation criterion;
6. an optional native structural-pruning rule that accepts only BIC-supported two- or three-child partitions satisfying minimum-size and visible-graph coherence constraints;
7. a K-layer representation for multiscale cluster evolution and an overlap-based diagnostic for studying cluster absorption and merging across K;

8. exact C–Python software regression gates, a controlled four-kernel benchmark, native pruning-sensitivity experiments, and a reproducible comparison with DBSCAN and KNN spectral clustering.

2 Related work

2.1 Shared-neighbour and Jarvis–Patrick clustering

Jarvis and Patrick introduced a clustering criterion based on shared near neighbours [6]. Later shared-nearest-neighbour (SNN) clustering methods used common neighbour counts to improve robustness to clusters of different shapes, sizes, and densities [7]. The present method is related to this family because reverse-neighbourhood intersections imply shared-neighbour evidence. However, shared-neighbour information is not used only as a static similarity score; it is embedded in a recursive graph-closure process.

2.2 Mutual, reciprocal, and reverse nearest-neighbour methods

Mutual KNN and reciprocal nearest-neighbour graphs have been used for clustering and outlier detection [8, 9, 13]. Reverse nearest-neighbour methods use incoming KNN links as density or representativeness signals [10]. In the present formulation, reverse-neighbour sets are used both as backward adjacency relations in a round-trip closure process and as the basis for a kernel-weighted popularity measure. This popularity score modulates the local visibility threshold that controls whether a recursive forward or backward transition is admitted.

A particularly close recent construction at the level of the popularity statistic is KNNG-CS [12]. It assigns every outgoing KNN edge a source-normalised softmax weight and defines node importance as the sum of the incoming weights. This shares with our formulation the primitive of distance-weighted reverse-neighbour popularity, but the subsequent operators and outputs are different. KNNG-CS computes node importance once, greedily selects the highest-scoring coreset representatives, and removes each selected point together with its reverse neighbours. The present method instead uses popularity to modulate visibility within a recursive forward–backward closure that produces cluster layers. Both formulations normalise the relative allocation of a source's votes, but KNNG-CS fixes the outgoing mass to one, whereas our source budget remains the geometry-dependent historical quantity $B_i = \sum_j 1/r_{ij}^2$. This preserves the original inverse-square operator exactly while allowing a controlled comparison of allocation shapes.

2.3 Density-based and spectral graph clustering

DBSCAN, OPTICS, and HDBSCAN define clusters through density connectivity and hierarchical density estimates [14–17]. Spectral clustering constructs a similarity graph and uses eigenvectors of the associated graph Laplacian to reveal cluster structure [5, 18]. The method proposed here is graph-based, but it is combinatorial and closure-driven: it does not require eigenvalue decomposition and it exposes a sequence of partitions as the neighbourhood size K varies.

2.4 Clustering methods for astronomical synthetic data

The evaluation of clustering methods on structured scientific datasets is closely related to the present work. In astronomical applications, clustering may be applied not only to spatial point clouds, but also to synthetic orbital data, extracted features, and reduced-dimensional representations. Marinho et al. [19] investigated clustering methods for astronomical orbital synthetic data under feature extraction and dimensionality reduction pipelines. That work provides a complementary application context for the present paper. While the previous study emphasised the empirical behaviour of clustering methods on orbital synthetic data, the current work focuses on the graph-topological mechanism of round-trip KNN closure and on its induced multiscale partition sequence.

3 Relational basis of the round-trip operator

The primitive idea can be stated for an arbitrary directed relation. Let

$$A = \{a_1, \ldots, a_N\}, \qquad B = \{b_1, \ldots, b_M\} \tag{2}$$

be finite sets and let

$$T \subseteq A \times B \tag{3}$$

be an asymmetric relation. The reflective composition induced by T is

$$R_T = T^{-1} \circ T \subseteq A \times A. \tag{4}$$

For $a \in A$, the reflective neighbourhood is

$$R_T(a) = T^{-1}(T(a)). \tag{5}$$

Equation (5) means that one first moves from a to the objects related to it in B and then returns to all objects in A that share those objects in B.

For a subset $S \subseteq A$, define

$$R_T(S) = \bigcup_{a \in S} R_T(a). \tag{6}$$

The recursive reflection is

$$R_T^0(S) = S, \tag{7}$$

$$R_T^{t+1}(S) = R_T(R_T^t(S)), \qquad t \geq 0. \tag{8}$$

The process saturates when

$$R_T^{s+1}(S) = R_T^s(S) \tag{9}$$

for the first time. The integer s is the saturation depth of S under R_T.

This relational view provides an abstract basis for the forward–backward mechanism used by the proposed graph operator. The algorithmic construction is obtained by instantiating the relation on a directed KNN graph.

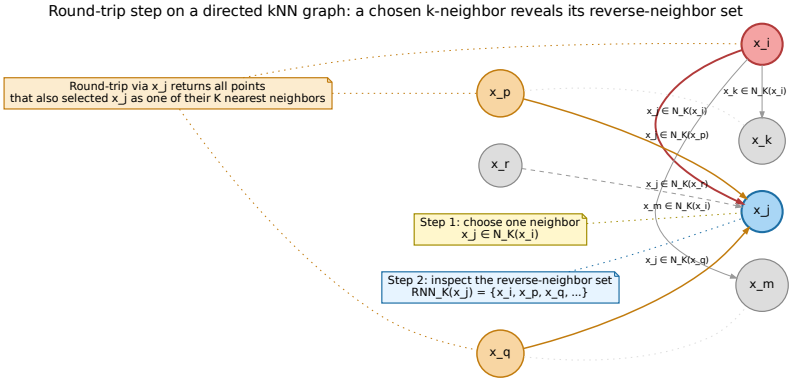


Fig. 1 Operational round-trip step on a directed KNN graph. A point x_i selects x_j as one of its K nearest neighbours. The selected neighbour x_j then reveals its reverse-neighbour set, i.e., all points that also selected x_j as one of their K nearest neighbours.

4 Directed KNN graphs and reverse neighbourhoods

Let

$$X = \{x_1, \ldots, x_N\} \subset \mathbb{R}^d \tag{10}$$

be a finite dataset with a metric $\delta : X \times X \to \mathbb{R}_{\geq 0}$. For each point x_i, let

$$\mathcal{N}_K^+(x_i) = \mathrm{KNN}_K(x_i) \tag{11}$$

denote the set of its K nearest proper neighbours. The directed KNN graph is

$$\mathcal{G}_K = (X, E_K), \tag{12}$$

where

$$(x_i, x_j) \in E_K \quad \Longleftrightarrow \quad x_j \in \mathcal{N}_K^+(x_i). \tag{13}$$

The reverse-neighbour set of x_j is

$$\mathcal{N}_K^-(x_j) = \mathrm{RNN}_K(x_j) = \{x_i \in X : x_j \in \mathcal{N}_K^+(x_i)\}. \tag{14}$$

Figure 1 illustrates the elementary operation. The forward step is a standard KNN edge. The backward step is not a Euclidean displacement; it is a query to the reverse-neighbour set of the selected point. This operation is the basic mechanism underlying the method presented in this paper, and provides a graph-theoretic realisation of the reflective composition $T^{-1} \circ T$.

The historical prototype also organised local popularity preferences into a leadership graph and then converted each of its weakly connected components into a rooted tree. Figure 3 records this provenance. These diagrams motivate the interpretation of reverse-neighbour popularity, but they are not an additional processing stage in the controlled vote-kernel benchmark reported below.

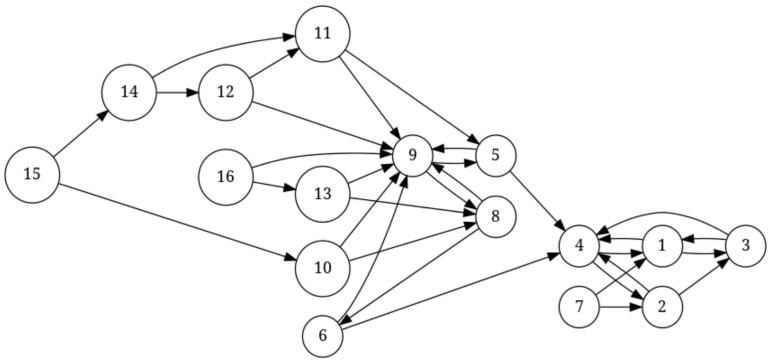


Fig. 2 Example of a directed KNN graph used to illustrate the proposed mechanism. Edges represent nearest-neighbour choices and arrowheads encode their direction. Nodes and edges are drawn uniformly in black because the red/blue convention in the archived rendering is undocumented and is not used algorithmically. The same graph supports reverse-neighbour retrieval and visibility-filtered round-trip expansion.

5 Vote-kernel popularity, visibility, and round-trip closure

The reference formulation adopts a popularity-modulated, visibility-filtered round-trip expansion. A point need not propagate through all of its K nearest-neighbour links: only links admitted by its local visibility threshold participate in the forward–backward closure. To reproduce the reference implementation exactly, linear and squared distances must be distinguished.

5.1 Distance representation and legacy radius

Let

$$r_{ij} = \delta(x_i, x_j), \qquad D_{ij} = r_{ij}^2. \tag{15}$$

The C implementation stores D_{ij}, rather than r_{ij}, in each KNN list. After the list is sorted, its stored radius is the largest retained squared distance,

$$R_K^{(2)}(x_i) = D_{i,(K)} = \max_{x_j \in \mathcal{N}_K^+(x_i)} D_{ij}. \tag{16}$$

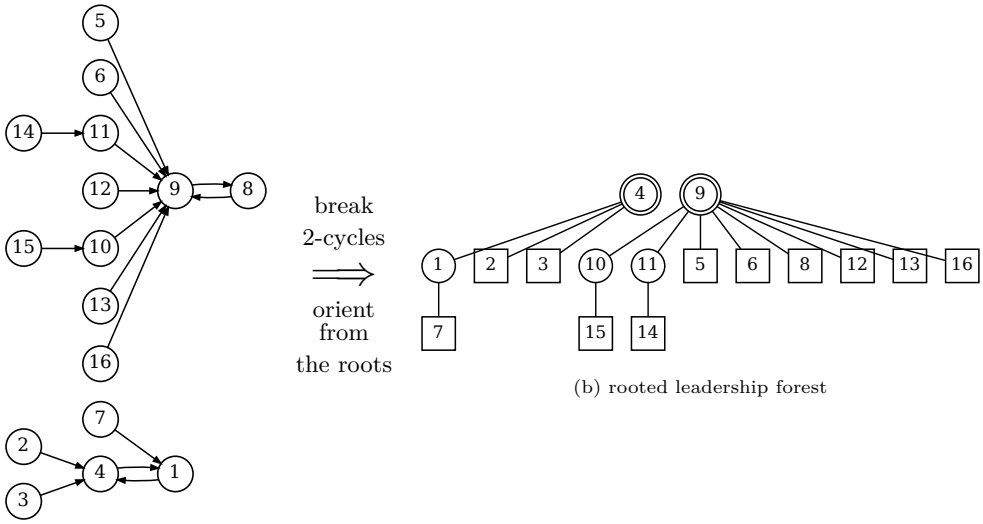
The corresponding linear scale is

$$R_K(x_i) = \sqrt{R_K^{(2)}(x_i)} = r_{i,(K)}. \tag{17}$$

5.2 Vote kernels and reverse-neighbour popularity

For each source point x_i, define the dimensionless neighbour coordinate

$$u_{ij} = \frac{r_{ij}}{R_K(x_i)}, \qquad x_j \in \mathcal{N}_K^+(x_i), \tag{18}$$



(a) directed leadership graph

(b) rooted leadership forest

Fig. 3 From the historical leadership graph to its rooted forest for the same 16-point KNN example. In (a), every vertex points to its locally selected leader; the graph has two weakly connected components, each containing a reciprocal leader pair, $1 \leftrightarrow 4$ and $8 \leftrightarrow 9$. Resolving each pair at its more popular vertex and displaying parent–child links away from roots 4 and 9 produces the two trees in (b). Vertex identifiers are preserved across panels, while nodes and edges are drawn uniformly in black so that colour carries no undocumented semantics. Double circles mark roots, single circles internal leaders, and squares leaves.

and compare the four allocation shapes

$$h(u) \in \left\{1, \frac{1}{u}, \frac{1}{u^2}, S_\alpha(u)\right\}, \tag{19}$$

where

$$S_\alpha(u) = \frac{1}{1 + \exp[\alpha(u-1)]}. \tag{20}$$

Here, $\alpha > 0$ controls the slope, and $R_K(x_i)$ supplies the local distance scale for every shape. Directly comparing the raw values 1, $1/r$, $1/r^2$, and $S_\alpha(r/R_i)$ would confound shape with units and arbitrary outgoing amplitude. We therefore retain the historical inverse-square vote mass of source i as the common budget

$$B_K(x_i) = \sum_{x_k \in \mathcal{N}_K^+(x_i)} \frac{1}{r_{ik}^2}, \tag{21}$$

and redistribute that budget according to the selected shape:

$$g_{i,h}(x_j) = B_K(x_i) \frac{h(u_{ij})}{\sum_{x_k \in \mathcal{N}_K^+(x_i)} h(u_{ik})}. \tag{22}$$

Thus $\sum_j g_{i,h}(x_j) = B_K(x_i)$ for every model, and the incoming popularity of x_j is

$$P_{K,g}(x_j) = \sum_{x_i:\, x_j \in \mathcal{N}_K^+(x_i)} g_{i,h}(x_j). \tag{23}$$

For $h(u) = 1/u^2$, Eq. (22) reduces algebraically to the historical edge vote,

$$g_{i,\mathrm{inv_r2}}(x_j) = B_K(x_i)\frac{(R_K(x_i)/r_{ij})^2}{\sum_{x_k \in \mathcal{N}_K^+(x_i)}(R_K(x_i)/r_{ik})^2} = \frac{1}{r_{ij}^2} = \frac{1}{D_{ij}}. \tag{24}$$

The implementation uses this final expression directly for the inverse-square branch, avoiding a normalise–denormalise round-off cycle. Exact duplicate observations are excluded because the shared historical budget is undefined at $r = 0$.

5.3 RMS visibility in squared-distance units

The unmodulated visibility threshold is the RMS of the stored squared distances,

$$V_{K,0}^{(2)}(x_i) = \left(\frac{1}{K}\sum_{x_j \in \mathcal{N}_K^+(x_i)} D_{ij}^2\right)^{1/2}. \tag{25}$$

Popularity then produces the effective threshold

$$V_{K,g}^{(2)}(x_i) = V_{K,0}^{(2)}(x_i)\left(1 + \frac{P_{K,g}(x_i)R_K^{(2)}(x_i)}{N}\right). \tag{26}$$

Equations (24)–(26) reproduce the historical inverse-square implementation exactly. The comparison models change only the within-source allocation in Eq. (22). Under a global coordinate rescaling $r \mapsto cr$, all four vote families scale as c^{-2} because $B_K \mapsto c^{-2}B_K$, while $R_K^{(2)} \mapsto c^2 R_K^{(2)}$; hence the popularity factor $P_{K,g}R_K^{(2)}$ is invariant for every model. Feature-wise preprocessing can still change the KNN graph and must therefore be specified and reused exactly.

5.4 Visible forward and backward steps

The visible forward neighbourhood of x_i is

$$\mathcal{V}_K^+(x_i) = \{x_j \in \mathcal{N}_K^+(x_i) : D_{ij} \le V_{K,g}^{(2)}(x_i)\}. \tag{27}$$

The visible backward neighbourhood of x_i is

$$\mathcal{V}_K^-(x_i) = \{x_\ell \in \mathcal{N}_K^-(x_i) : D_{\ell i} \le V_{K,g}^{(2)}(x_\ell)\}. \tag{28}$$

Algorithm 1 Visibility-filtered round-trip closure for one point

Require: directed KNN graph $\mathcal{G}_K$, vote model g, visibility thresholds $V_{K,g}^{(2)}$, point x_i

$C \leftarrow \{x_i\}$
repeat
  $C_{old} \leftarrow C$
  for all $x_q \in C_{old}$ **do**
    $C \leftarrow C \cup \mathcal{V}_K^+(x_q)$
    $C \leftarrow C \cup \mathcal{V}_K^-(x_q)$
  end for
until $C = C_{old}$
if $|C| > 1$ **then**
  return C
else
  return rejected singleton closure
end if

The backward condition is evaluated from the viewpoint of the reverse neighbour x_ℓ: a reverse-neighbour step is accepted only if $D_{\ell i}$ does not exceed the squared-distance visibility threshold of x_ℓ. The one-step visibility-filtered round-trip neighbourhood is therefore

$$\mathcal{R}_K^{\text{vis}}(x_i) = \mathcal{V}_K^+(x_i) \cup \mathcal{V}_K^-(x_i). \tag{29}$$

5.5 Recursive closure and singleton rejection

Starting from a seed point x_i, the recursive closure is

$$C_K^0(x_i) = \{x_i\}, \tag{30}$$

$$C_K^{t+1}(x_i) = C_K^t(x_i) \cup \bigcup_{x_q \in C_K^t(x_i)} \mathcal{R}_K^{\text{vis}}(x_q). \tag{31}$$

The process saturates when

$$C_K^{t+1}(x_i) = C_K^t(x_i). \tag{32}$$

The saturated closure is denoted

$$C_K^\infty(x_i) = C_K^s(x_i), \tag{33}$$

where s is the first saturation time.

A saturated closure is accepted as a cluster only when it contains more than one point:

$$|C_K^\infty(x_i)| > 1. \tag{34}$$

This singleton-rejection rule prevents isolated points or failed expansions from being reported as meaningful clusters. In the reported synthetic experiments, the accepted layers are subsequently checked to verify whether they cover the full dataset and therefore form proper partitions.

5.6 Optional structural pruning of saturated closures

The baseline closure has an intentionally strong connectivity semantics: one visible path is sufficient to join its endpoints. Structural pruning is an optional second stage, enabled in the native executable by `--structural-pruning`, that tests whether a large baseline component contains strongly supported internal populations. It is disabled by default, so the historical inverse-square regression and the four-kernel ablation remain unchanged. The native selector `--vote-kernel` accepts `uniform`, `inv_r`, `inv_r2`, or `sigmoid`; `inv_r2` remains the default, and the sigmoid slope is controlled by `--sigmoid-alpha` (default 8). The selected kernel supplies both the popularity-modulated visible graph and the popularity-ranked first mixture seed, while every subsequent pruning criterion is held fixed.

Let C be a baseline closure. It is considered for pruning only when

$$|C| \geq n_{\text{parent}} = \max\{50, \lceil 0.15N \rceil\}. \tag{35}$$

Each coordinate is standardised within C, and diagonal-covariance Gaussian mixtures with $m \in \{1, 2, 3, 4\}$ components are fitted by expectation–maximisation [20]. For dimension d, the implementation compares

$$\mathrm{BIC}_m = -2\ell_m + p_m \log |C|, \qquad p_m = m(2d + 1) - 1, \tag{36}$$

where ℓ_m is the maximised log likelihood and p_m counts mixture weights, means, and diagonal variances [21]. Models with $m > 1$ use five deterministic restarts. In the first restart, the initial centre is the point of greatest reverse-neighbour popularity and subsequent centres are chosen by farthest-first traversal; this is the operational role of leadership in the pruning stage. The remaining restarts use deterministic seeds, making repeated executions reproducible.

Let $m^* = \arg\min_m \mathrm{BIC}_m$. A candidate is retained only if

$$m^* \in \{2, 3\}, \qquad \mathrm{BIC}_1 - \mathrm{BIC}_{m^*} \geq 10, \tag{37}$$

and every proposed child contains at least

$$n_{\text{child}} = \max\{20, \lceil 0.05|C| \rceil\} \tag{38}$$

points. Thus, a four-component optimum is a conservative veto rather than an instruction to subdivide a curved or extensively fragmented closure.

Mixture membership alone does not determine the output. Each proposed child must also be coherent in the symmetrised visible KNN graph. A fragment smaller than n_{child} is reassigned to the alternative child with the greatest visible-edge support; the proposal is rejected if a child retains multiple large fragments or if a size constraint fails after regularisation. For an accepted labelling z_C, the crossing set is

$$E_C^{\text{cut}} = \left\{ \{i, j\} \in \overline{E}_{K,g}^{\text{vis}}[C] : z_C(i) \neq z_C(j) \right\}, \tag{39}$$

where $\overline{E}^{\mathrm{vis}}_{K,g}$ denotes the visible graph after disregarding edge orientation. The split is applied only when $|E^{\mathrm{cut}}_C| > 0$; its regularised children replace C in the final layer. Equivalently, this removes the accepted crossing visible connections before the component labels are rebuilt. Reference labels play no role in proposal generation, model selection, regularisation, or edge pruning.

6 Cluster layers and multiscale evolution

For a fixed K, the accepted saturated closures define a cluster layer

$$\mathcal{P}_K = \{C^{(K)}_1, \dots, C^{(K)}_{m_K}\}. \tag{40}$$

By varying K over a prescribed set of scales

$$K \in \{K_{\min}, K_{\min}+1, \dots, K_{\max}\}, \tag{41}$$

one obtains a multiscale sequence

$$\mathcal{P}_{K_{\min}}, \mathcal{P}_{K_{\min}+1}, \dots, \mathcal{P}_{K_{\max}}. \tag{42}$$

In the controlled experiments, each layer is an independent execution: $\mathcal{G}_K$, $R^{(2)}_K$, popularity, and visibility are constructed for that same value of K by invoking the supported interface with $K_0 = K$. This is an explicit experimental convention, not a claim that the intermediate layers of one raw $K_0 < K_{\max}$ process are numerically identical. In the latter mode, the neighbour list is allocated at $K_{\max}$ before the internal loop, so a stored construction-time radius may remain tied to $K_{\max}$ at its smaller layers. Independent execution makes every reported $\mathcal{P}_K$ a function of one stated neighbourhood scale and prevents this second scale from being implicit.

Increasing K tends to increase graph connectivity. Therefore, small clusters from lower-K layers are expected to be absorbed into larger clusters at higher K values, although the exact behaviour depends on the dataset and on the visibility rule.

6.1 Inter-layer overlap

The overlap between clusters in consecutive layers is

$$W^{(K,K')}_{ab} = |C^{(K)}_a \cap C^{(K')}_b|. \tag{43}$$

Useful normalisations include

$$F^{(K,K')}_{ab} = \frac{|C^{(K)}_a \cap C^{(K')}_b|}{|C^{(K)}_a|}, \tag{44}$$

and the Jaccard index

$$J^{(K,K')}_{ab} = \frac{|C^{(K)}_a \cap C^{(K')}_b|}{|C^{(K)}_a \cup C^{(K')}_b|}. \tag{45}$$

Inter-layer overlap is expected and informative: it describes how clusters from one scale are preserved, absorbed, or merged at another scale. This is distinct from intra-layer overlap, which would indicate that a given layer is not a proper partition.

6.2 Saturation scale

A beneficial side effect of the K-layer construction is that the neighbourhood scale can be selected from the saturation behaviour of the induced cluster-layer sequence. We denote by K_{sat} the first value of K for which the accepted layer becomes stable under further increases of the neighbourhood size, up to relabelling of cluster indices. Lower values of K often expose local fluctuations, microfragmentation, or noise-like components, whereas the saturated layer represents the dominant macro-scale aggregation structure.

Operationally, saturation can be detected by monitoring the overlap between consecutive layers. If each cluster in $\mathcal{P}_K$ maps entirely to a unique cluster in $\mathcal{P}_{K+1}$, and vice versa, the two layers are equivalent. More robustly, one may require the same equivalence to persist over a short stability window,

$$\mathcal{P}_K \equiv \mathcal{P}_{K+1} \equiv \cdots \equiv \mathcal{P}_{K+h}, \tag{46}$$

where h is a prescribed persistence parameter. The saturation scale can then be defined as

$$K_{\mathrm{sat}} = \min \left\{ K : \mathcal{P}_K \equiv \mathcal{P}_{K+1} \equiv \cdots \equiv \mathcal{P}_{K+h} \right\}. \tag{47}$$

Here, $\equiv$ denotes equality of cluster layers up to relabelling of cluster indices.

7 Implementation agreement and partition diagnostic

An important consistency check is whether each accepted layer $\mathcal{P}_K$ is a true partition of the dataset. For each K, this can be verified by comparing the sum of accepted cluster cardinalities with the cardinality of the union of all accepted cluster members:

$$\sum_{a=1}^{m_K} |C_a^{(K)}| = \left| \bigcup_{a=1}^{m_K} C_a^{(K)} \right|. \tag{48}$$

When the equality in Eq. (48) holds and the union contains all N data points, the accepted layer is not merely a cover of the dataset, but a proper partition:

$$\bigcup_{a=1}^{m_K} C_a^{(K)} = X, \qquad C_a^{(K)} \cap C_b^{(K)} = \varnothing \quad \text{for } a \neq b. \tag{49}$$

This property is important because the multiscale object produced by the algorithm is then a sequence of valid partitions, rather than a sequence of overlapping covers. Inter-layer overlap is still expected and informative, but intra-layer overlap would indicate an invalid layer representation. If singleton closures are rejected, the diagnostic also verifies whether any points were left outside the accepted layer.

Table 1 Unpruned inverse-square regression results for the variable-density blobs dataset. Each K was run independently with $K_0 = K$.

K	clusters	ARI	NMI	unclustered
2	595	0.0116	0.3991	0
3	277	0.0403	0.4491	0
4	99	0.2809	0.5716	0
5	18	0.6661	0.8017	0
6	6	0.7774	0.8941	0
8	4	0.7817	0.9035	0

If the left-hand side of Eq. (48) is larger than the right-hand side, at least one point belongs to more than one cluster in the same layer.

Before comparing vote kernels, the additive Python implementation was tested exclusively with the unpruned historical inverse-square model. The C executable and Python implementation received the same feature columns, datasets, and isolated values $K = 2, \ldots, 16$. Pointwise partitions were compared independently of cluster-number permutations, including an exact comparison of the unclustered masks. All 120 comparisons (eight datasets times 15 values of K) agreed exactly. This is a software regression gate: it establishes that the Python experiment reproduces the selected C baseline, but it is not by itself evidence that either implementation recovers an external reference partition or every earlier execution convention. The vote-kernel benchmark was permitted to proceed only after this gate passed. After native kernel selection was introduced, a second gate compared the unpruned C and Python partitions for the budget-matched shapes $h(u) = 1/u$, $h(u) = 1/u^2$, and $h(u) = S_\alpha(u)$ on the variable-density blobs at $K = 8$ and the bridge at $K = 6$. All six pointwise comparisons also agreed exactly before the pruned sensitivity figures were produced.

Table 1 reports a representative subset for the variable-density blobs dataset. Since all $N = 2000$ points are represented exactly once and the unclustered count is zero, these layers are proper partitions. ARI and NMI compare each partition with the known reference labels and are invariant under a permutation of cluster identifiers.

This diagnostic is different from inter-layer overlap. Inter-layer overlap is expected and informative; it describes how clusters from one scale are preserved, split, or absorbed at another scale.

8 Scale, popularity, visibility, and saturation

The formulation suggests an interpretation in terms of propagation, popularity, and selectivity. In the present graph setting, this balance is implemented through the neighbourhood scale K, the kernel-dependent reverse-neighbour popularity score $P_{K,g}(x_i)$, and the local visibility threshold $V^{(2)}_{K,g}(x_i)$. Small values of K preserve local selectivity but may fragment the graph. Larger values of K increase propagation, while popularity-modulated RMS visibility prevents all KNN links from being treated as equally admissible.

The base RMS visibility threshold in Eq. (25) acts as a pointwise scale estimate in squared-distance units, while Eq. (26) introduces a popularity-dependent enlargement. In dense regions the base threshold is small and expansion remains selective. Points receiving many influential incoming KNN links obtain higher popularity and may propagate through a larger visible neighbourhood. The kernel determines what constitutes an influential incoming link without changing the closure operator.

The singleton-rejection rule in Eq. (34) adds a second selectivity mechanism. A seed that cannot produce a nontrivial saturated closure is not reported as a cluster. Thus, the reported layer is determined not only by reachability in the directed KNN graph, but also by reverse-neighbour popularity, visibility-filtered propagation, and nontrivial closure size.

As K increases, the induced layers may pass from microfragmented local structures to stable macro-scale agglomerations. The saturation scale K_{sat} provides an empirical way to select the neighbourhood scale from the layer sequence itself, rather than fixing K solely as an external hyperparameter.

9 Experimental protocol

The controlled benchmark reuses the eight synthetic datasets already distributed with the implementation:

- Gaussian blobs with equal and variable densities;
- two moons and concentric circles;
- anisotropic Gaussian clusters;
- a uniform point cloud;
- two dense clusters connected by a sparse bridge;
- a Swiss-roll embedding.

The label column is used only for evaluation and is never included among the clustering features. The Swiss-roll labels are an artificial discretisation of a continuous manifold and its external scores are therefore interpreted descriptively. ARI and NMI are not reported for the uniform dataset because it has no nontrivial reference partition.

For every dataset, K ranges from 2 through 16. Each K is run independently with $K_0 = K$. The KNN graph is constructed once for a dataset–K pair and is reused unchanged by all four vote kernels. All models share the inverse-square source budget in Eq. (21) and therefore require strictly positive neighbour distances, so the input is checked for exact duplicate points before execution. Uniform, inverse-distance, inverse-square, and sigmoid refer to the four allocation shapes in Eq. (19); the sigmoid uses $\alpha = 8$. RMS visibility and singleton rejection are enabled for every model. Structural pruning is disabled throughout this 840-row comparison, preserving a controlled ablation in which only the within-source allocation changes.

The bridge benchmark varies the retained fraction of the existing labelled bridge points over $\{0, 0.10, 0.25, 0.50, 0.75, 1\}$. Core points are never subsampled. The two dense cores are considered merged when their dominant predicted cluster identifier is the same; this criterion is independent of the numerical names assigned to clusters.

The output is one long-format CSV. For each dataset, K, and vote model, it records:

- the dataset name, K, vote model, and complete model parameters;
- the number of clusters and unclustered points;
- ARI and NMI when a nontrivial reference partition is available [22, 23];
- the mean number of visible neighbours and mean visibility in squared-distance units;
- end-to-end runtime, including KNN-graph construction;
- the bridge-merger indicator for the parameterised bridge rows.

The same measured graph-construction time is included in each of the four model runtimes for a given dataset–K pair. This preserves end-to-end interpretation while ensuring that differences in graph construction cannot favour one vote kernel.

Structural pruning is evaluated separately in the native C executable under the historical inverse-square vote. Each case is run twice with identical coordinates and isolated K: once with the default unpruned closure and once with `--structural-pruning`. The selected cases are variable-density blobs at $K = 8$, the complete sparse bridge at $K \in \{6, 10\}$, and three preservation controls—circles at $K = 7$, two moons at $K = 10$, and anisotropic blobs at $K = 8$. The fixed implementation constants are one through four diagonal mixture components, five restarts for $m > 1$, a minimum BIC gain of 10, the parent and child sizes in Eqs. (35) and (38), and a four-component veto. The executable is passed `--dim 2`; the label column is read only afterwards to compute ARI and NMI. Complete native logs, layer files, input hashes, and a machine-readable parameter manifest accompany the figures.

To test whether the two principal corrections depend on the historical allocation shape, the variable-density blobs at $K = 8$ and the complete bridge at $K = 6$ are also run natively with the three budget-matched shapes $1/u$, $1/u^2$, and $S_\alpha(u)$, with $\alpha = 8$. Uniform allocation remains in the broader unpruned ablation but is excluded from this focused weighted-kernel check. For each kernel, the C baseline must first match the corresponding Python partition pointwise; the same native mixture, BIC, size, coherence, and pruning parameters are then applied without retuning.

9.1 Two-baseline comparison protocol

The final comparison uses the six datasets having nontrivial discrete references and two complementary methods: density-connected DBSCAN [14] and KNN-affinity spectral clustering [5, 18]. The uniform cloud has no reference partition, and the Swiss-roll bands are an artificial discretisation of a continuous manifold; both are therefore excluded from this table. No method-specific standardisation is applied: all three methods receive the same raw feature columns.

Structurally pruned RTKNNC uses the historical inverse-square vote and sweeps $K = 2, \ldots, 16$ in the native executable. DBSCAN sweeps $m \in \{3, 5, 10, 20\}$ for `min_samples`. For each m, let $d_i^{(m)}$ be the distance from x_i to its $(m-1)$st proper neighbour, because DBSCAN counts the query point itself. Its ϵ candidates are the 400 empirical quantiles of $\{d_i^{(m)}\}_{i=1}^N$ at $q = 0.0025, 0.005, \ldots, 1$. This label-free construction adapts the numerical grid to each dataset's coordinate scale. Spectral clustering uses a symmetrised KNN affinity with $k_g = 2, \ldots, 16$, ARPACK eigenvectors, and 20 k-means label-assignment restarts at random seed zero. Unlike RTKNNC and DBSCAN, it is supplied with the reference cluster count C^*.

For each method–dataset pair, the table reports the candidate with maximum ARI, then maximum NMI. Remaining ties prefer smaller K or k_g; DBSCAN then prefers fewer noise points, smaller m, and smaller ϵ. Thus, the result is explicitly a label-selected oracle envelope measuring what each fixed grid can represent, not a claim that ARI supplies an unsupervised deployment rule. DBSCAN's noise identifier -1 participates in ARI and NMI as a predicted label; $\widehat{C}$ excludes it and n_0 reports its cardinality. The complete 9,780-row sweep, selected rows, software versions, input hashes, and parameter manifest are generated with the article artifacts.

10 Vote-model benchmark results

The complete sweep contains 840 observations: 480 rows from eight datasets, 15 values of K, and four kernels, plus 360 rows from six bridge fractions, 15 values of K, and four kernels. No observation contained an unclustered point. Among the 210 matched dataset–scale configurations, the number of clusters differed across kernels in 60 configurations; ARI and NMI differed in 57 labelled configurations. The mean number of visible neighbours and the mean visibility both differed in all 210 configurations. Thus, the allocation shape frequently changes the internal visibility dynamics, although the resulting external optimum can remain stable.

For anisotropic clusters, equal-density blobs, circles, and moons, every kernel attained ARI=NMI=1 within the tested K range. In this unpruned ablation, all kernels also attained the same best result for variable-density blobs (ARI 0.7817, NMI 0.9035, $K = 8$) and for the labelled bridge dataset (ARI 0.8083, NMI 0.8182, $K = 6$). These are baseline optima, not the final results of the structural-pruning enhancement reported in Sect. 11. For the discretised Swiss roll, the best scores were low, as expected for an artificial banding of a continuous manifold. All optima occurred at $K = 4$: uniform, inverse-distance, inverse-square, and sigmoid allocation attained ARI/NMI 0.1559/0.5160, 0.1544/0.5141, 0.1528/0.5131, and 0.1558/0.5153, respectively.

For the coloured comparison figures only, a common representative K is selected for each labelled dataset by maximising the mean ARI across the four kernels, with ties resolved in favour of the smaller K. The same selected K is then used in every predicted panel, so the graphical comparison does not grant a different scale to each kernel. This visualisation rule does not filter the long-format results, which retain all $K = 2, \ldots, 16$ observations.

Figure 4 provides a representative non-convex case in which all four kernels recover the reference partition exactly. The more difficult variable-density case is reserved for the focused, pruned three-kernel mosaic in Fig. 6.

The cluster count alone can conceal this failure. At $K = 6$, the six reported components do not include the compact overlapping Gaussian as an independent cluster: one component contains all 400 of its points together with 390 of the 400 points from the diffuse Gaussian. The two additional components are small fragments. At $K = 8$, the merged component contains all 400 compact points and 399 diffuse points. No tested value $K = 2, \ldots, 16$ in the *unpruned* four-kernel sweep recovers the five-class reference partition: smaller K fragments the point cloud before the overlapping distributions separate, whereas larger K connects them. This observation diagnoses

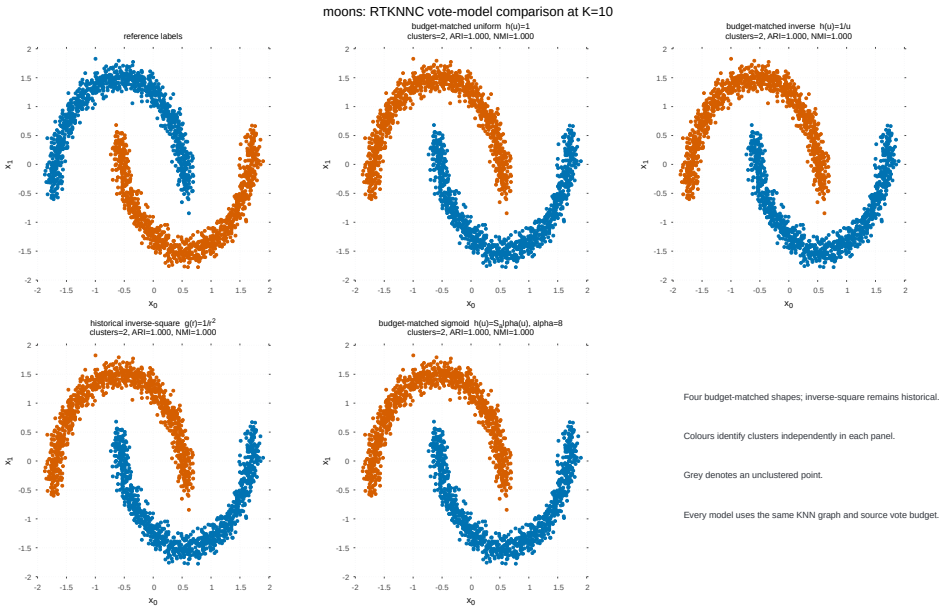


Fig. 4 Reference labels and the four predicted partitions for the two-moons dataset at the common representative scale $K = 10$: budget-matched uniform $h(u) = 1$, inverse-distance $h(u) = 1/u$, historical inverse-square $h(u) = 1/u^2$, and local sigmoid $S_\alpha(u)$ with $\alpha = 8$. Every source casts the common budget in Eq. (21); the panels differ only in its allocation among outgoing neighbours. Every vote model recovers the two non-convex components exactly (ARI = NMI = 1). The GNUplot rendering explicitly labels both coordinate axes; cluster colours are local to each panel and do not encode a cross-panel label correspondence.

the connectivity closure alone; it is precisely the failure corrected by the label-free native pruning result in Fig. 6.

10.1 Parameterised bridge transition

Table 2 reports the first value of K at which the two dense cores share a dominant predicted cluster. No kernel merged the disconnected cores when all bridge points were removed. At bridge fraction 0.10, the historical inverse-square allocation merges one scale later than the other three; the boundaries coincide at the remaining tested fractions. This is direct evidence that the budget-matched shapes are not equivalent implementations.

For the complete bridge at $K = 6$, the two reported components also do not correspond to the three reference labels. One contains all 900 points of the left core, all 200 bridge points, and six peripheral points from the right core; the other contains the remaining 894 right-core points. The bridge has therefore already been absorbed by the left component even though the two dense cores have not yet fully merged.

Table 2 First K producing a merger of the two dense bridge cores for each of the four vote kernels. A value > 16 means that no merger occurred in the tested range.

bridge fraction	$h(u) = 1$	$h(u) = 1/u$	$h(u) = 1/u^2$	$S_\alpha(u)$
0.00	> 16	> 16	> 16	> 16
0.10	5	5	6	5
0.25	6	6	6	6
0.50	6	6	6	6
0.75	7	7	7	7
1.00	7	7	7	7

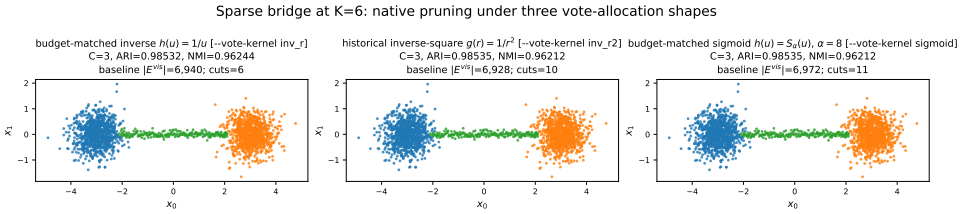


Fig. 5 Native structural pruning of the complete sparse bridge at $K = 6$ under the three weighted allocation shapes. The panels show budget-matched $h(u) = 1/u$, historical $h(u) = 1/u^2$, and budget-matched local sigmoid $S_\alpha(u)$ with $\alpha = 8$, respectively. Every model produces three clusters. Inverse-square and sigmoid allocation coincide pointwise (ARI 0.98535, NMI 0.96212), while inverse-distance differs from them at four points after label alignment (ARI 0.98532, NMI 0.96244). The three models remove 6, 10, and 11 crossing visible connections, and their annotated native baseline visible-edge counts are 6940, 6928, and 6972, respectively. Each panel comes from a separate native execution whose recorded command contains the displayed `--vote-kernel` token; the equal second and third scores reflect a pointwise-identical final partition despite distinct baseline graphs, not reuse of a layer file. The reference panel is intentionally omitted; reference labels are not available to the clustering or pruning stages.

11 Native structural-pruning results

Table 3 separates the native pruning experiment from the four-kernel sweep. The variable-density case changes from four to five clusters at $K = 8$, with ARI increasing from 0.7817 to 0.9627. The accepted split removes 12 crossing visible connections. The result is not a relabelling of the reference partition: the five inferred cluster sizes are 375, 400, 400, 401, and 424, so the remaining disagreement is reported rather than hidden by the correct cluster count.

For the complete sparse bridge, pruning recovers three components at both tested scales. At $K = 6$, ARI increases from 0.8083 to 0.9853 after 10 crossing connections are pruned. At $K = 10$, the unpruned closure has already collapsed to one component, but a three-child split removes 32 crossing connections and reaches ARI 0.9865. Circles, moons, and anisotropic blobs serve as preservation controls: no split is accepted, no connection is removed, and their ARI remains exactly 1.

Table 3 Native inverse-square baseline and structural-pruning results. C^* is the number of reference classes, C_0 and C_p are the unpruned and pruned cluster counts, and $|E^{\mathrm{cut}}|$ is the number of crossing visible connections removed. Labels are used only for the reported ARI and C^*.

dataset	K	C^*	C_0	ARI_0	C_p	ARI_p	$\lvert E^{\mathrm{cut}}\rvert$
Variable-density blobs	8	5	4	0.7817	5	0.9627	12
Sparse bridge	6	3	2	0.8083	3	0.9853	10
Sparse bridge	10	3	1	0.0000	3	0.9865	32
Circles	7	2	2	1.0000	2	1.0000	0
Two moons	10	2	2	1.0000	2	1.0000	0
Anisotropic blobs	8	3	3	1.0000	3	1.0000	0

11.1 Pruning sensitivity to the vote kernel

Figures 5 and 6 address whether the successful split is an artefact of the historical inverse-square vote. For the variable-density blobs, all three weighted allocation shapes start from four unpruned components and produce the same five-cluster partition after pruning: ARI 0.9627, NMI 0.9614, and 12 removed connections. The three partitions are pointwise identical up to label permutation. Their mean visible-neighbour counts are not identical (4.7010, 4.6825, and 4.7350, respectively), confirming that the kernels alter the visible graph. In the companion Python edge audit, whose partitions pass the native agreement gate, the inverse-square directed edge set differs from the inverse-distance and sigmoid sets by 89 and 269 visible edges, respectively. These changes remain inside the same connected components, and the accepted pruning split therefore coincides at this scale.

The bridge result is also stable in its scientific conclusion. Every kernel starts from two components and returns the intended three, with ARI between 0.98532 and 0.98535. Inverse-square and sigmoid allocation induce the same partition, while inverse-distance differs by four peripheral assignments after optimal label alignment. They remove 10, 11, and 6 crossing connections, respectively, when listed as inverse-square, sigmoid, and inverse-distance. In the companion Python edge audit, the inverse-square baseline visible edge set differs by 56 and 190 directed edges from inverse-distance and sigmoid voting, respectively. The equality of cluster counts is therefore supported by near-identical external partitions, not used as a substitute for them.

12 Comparison with DBSCAN and spectral clustering

Table 4 reports the oracle envelope defined in Sect. 9. All three methods attain ARI = NMI = 1 on anisotropic blobs, equal-density blobs, circles, and two moons. These controls therefore do not distinguish the methods once each receives its best grid candidate.

The two connectivity-stress cases do. On the variable-density blobs, RTKNNC returns the intended five clusters with ARI 0.9627 and NMI 0.9614. Spectral clustering, despite being supplied with $C^* = 5$, reaches ARI 0.9569 and NMI 0.9581.

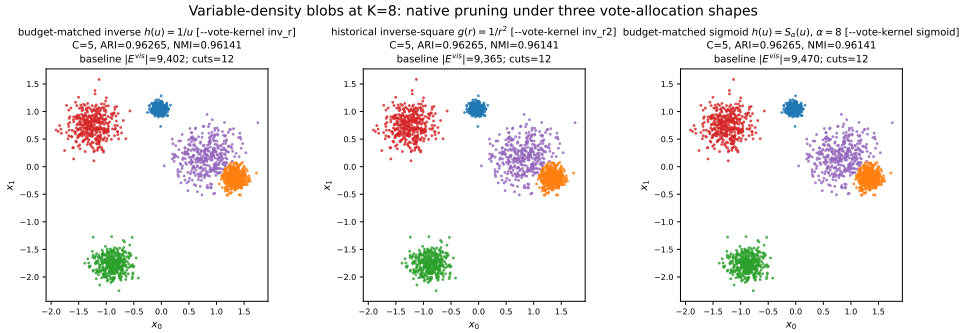


Fig. 6 Native structural pruning of the variable-density blobs at $K = 8$ under the three weighted allocation shapes. The three panels show, from left to right, budget-matched $h(u) = 1/u$, historical $h(u) = 1/u^2$, and budget-matched local sigmoid $S_\alpha(u)$ with $\alpha = 8$. All produce the same pointwise five-cluster partition (ARI 0.96265, NMI 0.96141) after removing 12 crossing visible connections. The panel annotations expose the distinct native baseline graphs: $|E^{\text{vis}}| = 9402$, 9365, and 9470 directed edges, respectively. Reference labels are omitted from the mosaic and are used only for colour alignment and external scores; each unpruned C partition was first verified pointwise against its Python kernel definition.

Table 4 Best label-selected grid result for pruned RTKNNC, DBSCAN, and KNN spectral clustering. Each score cell reports ARI/NMI; bold denotes the highest ARI in the row, including ties. m is DBSCAN's `min_samples`, k_g is the spectral affinity-neighbour count, $\widehat{C}$ excludes DBSCAN's noise label, and n_0 is its noise count. Spectral clustering receives C^*, whereas RTKNNC and DBSCAN do not. These are oracle envelopes over the fixed grids, not label-free parameter-selection results.

		RTKNNC + pruning		DBSCAN		Spectral	
dataset	C^*	setting	ARI/NMI	setting	ARI/NMI	setting	ARI/NMI
Anisotropic blobs	3	$K = 8,\ \widehat{C} = 3$	**1.0000/1.0000**	$\epsilon = 1.128,\ m = 3;\ \widehat{C} = 3,\ n_0 = 0$	**1.0000/1.0000**	$k_g = 6,\ \widehat{C} = 3$	**1.0000/1.0000**
Equal-density blobs	4	$K = 7,\ \widehat{C} = 4$	**1.0000/1.0000**	$\epsilon = 0.8163,\ m = 20;\ \widehat{C} = 4,\ n_0 = 0$	**1.0000/1.0000**	$k_g = 4,\ \widehat{C} = 4$	**1.0000/1.0000**
Variable-density blobs	5	$K = 8,\ \widehat{C} = 5$	**0.9627/0.9614**	$\epsilon = 0.05965,\ m = 5;\ \widehat{C} = 15,\ n_0 = 204$	0.7916/0.8044	$k_g = 6,\ \widehat{C} = 5$	0.9569/0.9581
Sparse bridge	3	$K = 11,\ \widehat{C} = 3$	**0.9865/0.9632**	$\epsilon = 0.1089,\ m = 3;\ \widehat{C} = 14,\ n_0 = 116$	0.8140/0.7422	$k_g = 6,\ \widehat{C} = 3$	0.9487/0.8993
Circles	2	$K = 7,\ \widehat{C} = 2$	**1.0000/1.0000**	$\epsilon = 0.1763,\ m = 3;\ \widehat{C} = 2,\ n_0 = 0$	**1.0000/1.0000**	$k_g = 5,\ \widehat{C} = 2$	**1.0000/1.0000**
Two moons	2	$K = 10,\ \widehat{C} = 2$	**1.0000/1.0000**	$\epsilon = 0.2423,\ m = 3;\ \widehat{C} = 2,\ n_0 = 0$	**1.0000/1.0000**	$k_g = 6,\ \widehat{C} = 2$	**1.0000/1.0000**

DBSCAN reaches ARI 0.7916 and NMI 0.8044, while its score-maximising candidate fragments the data into 15 clusters and marks 204 points as noise. On the sparse bridge, RTKNNC reaches ARI 0.9865 and NMI 0.9632, compared with 0.9487/0.8993 for spectral clustering and 0.8140/0.7422 for DBSCAN. The selected DBSCAN bridge candidate similarly has 14 clusters and 116 noise points. The RTKNNC bridge envelope selects $K = 11$ by a marginal ARI advantage over $K = 10$; the focused $K = 6$ experiment remains useful because it isolates the earliest corrected three-component partition under the three vote kernels.

The comparison supports a narrower conclusion than universal superiority. It shows that the structural correction is competitive on the four standard controls and retains an advantage on the two synthetic structures for which visible connectivity is deliberately difficult. Repeated stochastic realisations, higher-dimensional data, and a label-free parameter-selection study remain necessary.

13 Discussion

The method should be positioned carefully. It is not a first use of KNN graphs for clustering, nor a replacement for all density-based methods. Its distinctive aspect is the recursive use of forward KNN selection and reverse-neighbour retrieval under a popularity-modulated RMS visibility filter, followed by singleton rejection and saturation analysis. The optional enhancement adds conservative, leadership-seeded structural pruning after closure rather than replacing the graph operator. This places the method between shared-neighbour clustering, reverse-neighbour graph reasoning, closure-based graph clustering, and local model-based partitioning.

The baseline method is an algorithmic and topological construction rather than the minimisation of an explicit objective function. The benchmark supports this distinction: changing only the vote allocation altered the cluster count in 60 of 210 matched configurations and altered visibility in every configuration, yet several datasets shared the same externally optimal partition across all kernels. The allocation shape controls the distribution of incoming evidence, whereas the round-trip graph and its closure determine how visible evidence propagates. Structural pruning is deliberately reported as a separate stage because its BIC criterion introduces an explicit local statistical model.

The inverse-square kernel retains a special implementation-level status: it is the historical rule and is preserved exactly. A raw comparison against 1, $1/r$, and a bounded sigmoid would, however, mix kernel shape with incompatible units and outgoing amplitudes; on these data, the raw sigmoid was consequently almost indistinguishable from uniform voting. The source-budget-matched family removes that confound by conserving $B_K(x_i)$ for every source and changing only how that mass is distributed among its K outgoing edges. It also makes $P_{K,g}R_K^{(2)}$ invariant under global coordinate rescaling for every model, not just inverse-square. Feature-wise transformations can still change the neighbour graph, so preprocessing remains part of the method specification.

Within the fixed coordinates of the two difficult cases, however, the pruning conclusion is not specific to inverse-square voting. This should not be read as kernel equivalence. For the variable-density blobs, the three kernels produce pointwise-identical final partitions even though their native mean visible-neighbour counts differ and the companion audit finds hundreds of kernel-dependent directed edges. Those edge changes occur within the same symmetrised connected components, and the mixture stage accepts the same geometric subdivision. For the bridge, the stability is only approximate: inverse-distance voting changes four assignments relative to the pointwise- identical inverse-square and sigmoid results, a distinction hidden by three-decimal ARI and NMI. Thus, the matching blob scores identify an empirical topological stability plateau, not redundant kernel implementations, while the bridge demonstrates that small kernel effects remain measurable. This does not establish general kernel invariance; it shows that the reported correction survives the most immediate alternative-kernel challenge without retuning the pruning rule.

The bridge result should also be interpreted narrowly. At retained bridge fraction 0.10, uniform, inverse-distance, and sigmoid allocation first merge the dense cores at $K = 5$, whereas historical inverse-square first merges them at $K = 6$. This

observed boundary difference confirms that equal results at the complete-bridge reference scale do not imply kernel equivalence. A denser sweep over bridge geometry, noise, separation, and sigmoid slope may expose further transition differences.

13.1 Connectivity failure and structural correction

The observed failures follow from the topology of the closure operator. For a fixed K and vote kernel g, define the visible directed edge set

$$E_{K,g}^{\mathrm{vis}} = \left\{(i,j) : x_j \in \mathcal{N}_K^+(x_i),\ D_{ij} \le V_{K,g}^{(2)}(x_i)\right\}. \tag{50}$$

Forward expansion traverses outgoing edges in this graph and backward expansion traverses incoming edges. Consequently, apart from the explicit singleton policy, the *unpruned* saturated closures are the connected components obtained after disregarding the orientation of the visible edges. The baseline can therefore recover a reference partition exactly only at a scale for which its classes form separate components of this symmetrised visible graph. The enhanced method relaxes that restriction in a controlled way: Eqs. (37)–(39) permit a large component to be divided only when independent mixture and graph-coherence tests agree.

This condition explains why nonconvexity itself is not fatal. Moons and circles are nonconvex, but a geometric gap prevents visible paths between their reference components at an appropriate value of K. The variable-density example is different: the compact and diffuse right-hand Gaussians overlap in the same region of the observed space. Cross-class nearest-neighbour edges are then unavoidable, and one admissible chain places both latent classes in the same baseline closure. Decreasing K removes edges but first produces microclusters; increasing K strengthens the connection. Structural pruning supplies the missing modelling assumption locally: at $K = 8$, BIC supports a two-population subdivision of the merged closure and the visible graph supports both children. This recovers five clusters with ARI 0.9627, while the residual error appropriately reflects that overlapping coordinates do not define a uniquely perfect latent boundary.

The sparse bridge creates the analogous phenomenon through a filament rather than direct overlap. Its left endpoint reaches vertices that already belong to the giant left-core component, whereas the closest right-end vertices remain in small peripheral components until the next scale. Once a visible path reaches a giant component, recursive forward–backward expansion absorbs the entire path; a single cross-boundary connection is sufficient in the baseline. The native stage identifies three supported populations and prunes 10 crossing connections at $K = 6$, preserving the bridge as a third structure with ARI 0.9853. The Swiss roll presents the complementary case: its reference bands discretise one continuous manifold, so selective visibility yields many local components rather than the prescribed bands, while less selective scales eventually connect across them. The four-component veto and graph-coherence test are intended to protect such curved or fragmented structures, but a systematic manifold study remains necessary.

Thus, the baseline output is a partition of visible KNN connectivity, whereas the enhanced output is that partition after a conservative local model-based test of

its large components. Neither is guaranteed to reconstruct every externally supplied label partition. The distinction matters scientifically: the four-kernel sweep measures vote sensitivity of the original closure, while the native comparison measures the incremental effect of structural pruning.

The present study has several limitations:

1. the synthetic results use one deterministic realisation of each dataset family;
2. the sigmoid experiment fixes $\alpha = 8$ rather than sweeping its slope, and the focused three-kernel pruning comparison is restricted to the two difficult reference cases;
3. the bridge benchmark varies bridge density but not width, curvature, core separation, or noise;
4. the pruning thresholds and diagonal-covariance family are fixed and have been evaluated here on six primary dataset–scale cases;
5. overlapping latent classes may remain only approximately identifiable, and broader tests are needed to quantify false splits on curved, heavy-tailed, and high-dimensional data;
6. the empirical runtime measurements are not a substitute for a formal complexity analysis;
7. the external comparison is restricted to DBSCAN and KNN spectral clustering on six two-dimensional datasets and uses reference labels to select each grid optimum; label-free selection, repeated realisations, and comparisons with SNN, mutual-KNN, and hierarchical-density methods remain open.

14 Conclusion

We presented a clustering method based on popularity-modulated, visibility-filtered round-trip closure in directed KNN graphs, separated its topological operator from four source-budget-matched vote-allocation shapes, preserved the historical inverse-square rule exactly, and added optional native structural pruning for overly connected closures. A 120-case inverse-square C–Python agreement test served as a software regression gate, and the unpruned 840-row sweep showed that the kernel often changes visibility and sometimes the induced partition. The native enhancement then corrected the central connectivity failures: variable-density blobs changed from four to five clusters with ARI $0.7817 \rightarrow 0.9627$, and the bridge reached three clusters with ARI 0.9853 at $K = 6$ and 0.9865 at $K = 10$. It accepted no split on three already correct controls, whose ARI remained 1. At the two difficult reference scales, inverse-distance, inverse-square, and sigmoid voting all preserved the corrected cluster counts with essentially identical scores, and their six unpruned native partitions passed pointwise C–Python agreement. A 9,780-candidate oracle-envelope comparison then showed that pruned RTKNNC, DBSCAN, and KNN spectral clustering all recover the four control datasets perfectly, while RTKNNC attains the best ARI on the variable-density blobs (0.9627) and sparse bridge (0.9865), including against spectral clustering given the reference cluster count. These results show that vote weighting, visible connectivity, and structural correction are distinct experimental axes. Broader replication is still required for the fixed pruning thresholds, high-dimensional data,

alternative covariance models, label-free scale selection, and additional graph-based and hierarchical-density baselines.

Acknowledgements. To be completed.

Declarations.

- Funding: To be completed.
- Conflict of interest/Competing interests: The authors declare that they have no competing interests. To be confirmed.
- Ethics approval and consent to participate: Not applicable.
- Consent for publication: Not applicable.
- Data availability: Synthetic datasets and scripts will be made available in a public repository upon publication. To be completed.
- Materials availability: Not applicable.
- Code availability: The implementation will be made available in a public repository upon publication. To be completed.
- Author contribution: To be completed.

References

[1] Cover, T.M., Hart, P.E.: Nearest neighbor pattern classification. IEEE Transactions on Information Theory **13**(1), 21–27 (1967) https://doi.org/10.1109/TIT.1967.1053964

[2] Loftsgaarden, D.O., Quesenberry, C.P.: A nonparametric estimate of a multivariate density function. The Annals of Mathematical Statistics **36**(3), 1049–1051 (1965) https://doi.org/10.1214/aoms/1177700079

[3] Bentley, J.L.: Multidimensional binary search trees used for associative searching. Communications of the ACM **18**(9), 509–517 (1975) https://doi.org/10.1145/361002.361007

[4] Friedman, J.H., Bentley, J.L., Finkel, R.A.: An algorithm for finding best matches in logarithmic expected time. ACM Transactions on Mathematical Software **3**(3), 209–226 (1977) https://doi.org/10.1145/355744.355745

[5] Luxburg, U.: A tutorial on spectral clustering. Statistics and Computing **17**(4), 395–416 (2007) https://doi.org/10.1007/s11222-007-9033-z

[6] Jarvis, R.A., Patrick, E.A.: Clustering using a similarity measure based on shared near neighbors. IEEE Transactions on Computers **C-22**(11), 1025–1034 (1973) https://doi.org/10.1109/T-C.1973.223640

[7] Ertöz, L., Steinbach, M., Kumar, V.: Finding clusters of different sizes, shapes, and densities in noisy, high dimensional data. In: Proceedings of the 2003 SIAM International Conference on Data Mining, pp. 47–58. SIAM, Philadelphia, PA, USA (2003). https://doi.org/10.1137/1.9781611972733.5

[8] Brito, M.R., Chávez, E.L., Quiroz, A.J., Yukich, J.E.: Connectivity of the mutual k-nearest-neighbor graph in clustering and outlier detection. Statistics & Probability Letters **35**(1), 33–42 (1997) https://doi.org/10.1016/S0167-7152(96)00213-1

[9] Gowda, K.C., Krishna, G.: Agglomerative clustering using the concept of mutual nearest neighbourhood. Pattern Recognition **10**(2), 105–112 (1978) https://doi.org/10.1016/0031-3203(78)90018-3

[10] Chowdhury, S., Amorim, R.C.: An efficient density-based clustering algorithm using reverse nearest neighbour. arXiv preprint arXiv:1811.07615 (2018) arXiv:1811.07615 [cs.LG]

[11] Xie, W.-B., Lee, Y.-L., Wang, C., Chen, D.-B., Zhou, T.: Hierarchical clustering supported by reciprocal nearest neighbors. arXiv preprint arXiv:1907.04915 (2019) arXiv:1907.04915 [cs.LG]

[12] Liu, Y., Zhang, L., Xie, J., Wang, M., Yu, J.X., Cui, J.: Efficient Coreset Selection via K-Nearest Neighbor Graphs. Preprint, submitted 17 August 2026 (2026). https://arxiv.org/abs/2608.16270

[13] Sardana, D., Bhatnagar, V.: Graph clustering using mutual k-nearest neighbors. In: Advanced Machine Learning Technologies and Applications. Communications in Computer and Information Science, vol. 488, pp. 42–51. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-09912-5_4

[14] Ester, M., Kriegel, H.-P., Sander, J., Xu, X.: A density-based algorithm for discovering clusters in large spatial databases with noise. In: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining. KDD'96, pp. 226–231. AAAI Press, Menlo Park, CA, USA (1996)

[15] Ankerst, M., Breunig, M.M., Kriegel, H.-P., Sander, J.: Optics: Ordering points to identify the clustering structure. In: Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data, pp. 49–60. ACM, New York, NY, USA (1999). https://doi.org/10.1145/304181.304187

[16] Campello, R.J.G.B., Moulavi, D., Sander, J.: Density-based clustering based on hierarchical density estimates. In: Pei, J., Tseng, V.S., Cao, L., Motoda, H., Xu, G. (eds.) Advances in Knowledge Discovery and Data Mining. Lecture Notes in Computer Science, vol. 7819, pp. 160–172. Springer, Berlin, Heidelberg (2013). https://doi.org/10.1007/978-3-642-37456-2_14

[17] Campello, R.J.G.B., Moulavi, D., Zimek, A., Sander, J.: Hierarchical density estimates for data clustering, visualization, and outlier detection. ACM Transactions on Knowledge Discovery from Data **10**(1), 5–1551 (2015) https://doi.org/10.1145/2733381

[18] Ng, A.Y., Jordan, M.I., Weiss, Y.: On spectral clustering: Analysis and an algorithm. In: Advances in Neural Information Processing Systems, vol. 14, pp. 849–856 (2002)

[19] Marinho, E.P., Junior, N.C., Breve, F.A., Ranieri, C.M.: Clustering astronomical orbital synthetic data using advanced feature extraction and dimensionality reduction techniques. Neural Computing and Applications **38** (2026)

[20] Dempster, A.P., Laird, N.M., Rubin, D.B.: Maximum likelihood from incomplete data via the EM algorithm. Journal of the Royal Statistical Society: Series B (Methodological) **39**(1), 1–38 (1977) https://doi.org/10.1111/j.2517-6161.1977.tb01600.x

[21] Schwarz, G.: Estimating the dimension of a model. The Annals of Statistics **6**(2), 461–464 (1978) https://doi.org/10.1214/aos/1176344136

[22] Hubert, L., Arabie, P.: Comparing partitions. Journal of Classification **2**(1), 193–218 (1985) https://doi.org/10.1007/BF01908075

[23] Vinh, N.X., Epps, J., Bailey, J.: Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. Journal of Machine Learning Research **11**, 2837–2854 (2010)